# 不動産業BPO詳細設計

**スコープ**: 賃貸管理（PM） + 売買仲介 + 賃貸仲介
**パイロット**: 管理物件50-500戸の中小不動産管理会社
**制約**: REINS（レインズ）は閉鎖NW — 公開データ＋自社蓄積で対応
**目標**: 4週間で不動産BPO全8モジュール稼働

---

## マイクロエージェント対応表

| 設計書での呼称 | 実装関数名 | ファイル |
|---|---|---|
| OCR/テキスト抽出 | run_document_ocr | workers/micro/ocr.py |
| 構造化抽出 | run_structured_extractor | workers/micro/extractor.py |
| ルール照合 | run_rule_matcher | workers/micro/rule_matcher.py |
| コスト計算 | run_cost_calculator | workers/micro/calculator.py |
| コンプライアンスチェック | run_compliance_checker | workers/micro/compliance.py |
| 文書生成 | run_document_generator | workers/micro/generator.py |
| バリデーション | run_output_validator | workers/micro/validator.py |
| SaaS読取 | run_saas_reader | workers/micro/saas_reader.py |
| SaaS書込 | run_saas_writer | workers/micro/saas_writer.py |
| メッセージ生成 | run_message_drafter | workers/micro/message.py |
| 差分検出 | run_diff_detector | workers/micro/diff.py |
| PDF生成 | run_pdf_generator | workers/micro/pdf_generator.py |

---

# 0. エグゼクティブサマリー

## なぜ不動産BPOか

```
不動産業市場: 46兆円/年
管理会社の課題:
  ・家賃回収/督促が属人化 → 滞納率が全国平均5-7%で高止まり
  ・査定が経験と勘 → ±20%のブレが当たり前。売主を逃す or 安値受託
  ・契約書を1件1-3時間かけて手作業 → 宅建士のボトルネック
  ・修繕依頼→業者手配→見積→承認のリードタイム2-3週間
 → 全部AIで代行 = BPOコア¥250,000 + 追加モジュール¥100,000/個の価値

事業所数: 3.2万社（管理業登録事業者）
ターゲット: 管理戸数50-500戸の中小管理会社（約1.5万社）
```

## 8モジュール全体像

```
┌──────────────────────────────────────────────────────────────┐
│          不動産BPO（BPOコア¥250,000 + 追加モジュール¥100,000/個）          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ★ Phase A（Week 1-2）: 最優先                                 │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ① 物件査定AI       │  │ ② 契約書AI自動生成              │  │
│  │ 3手法査定→加重平均   │  │ 35条/37条書面+特約自動生成    │  │
│  │ ¥8-20万/月の価値    │  │ ¥3-8万/月の価値               │  │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase B（Week 3）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ③ 賃料・収支管理     │  │ ④ 送金・入金管理               │  │
│  │ 入金消込→督促→送金   │  │ 敷金台帳+インボイス対応       │  │
│  │ ✅ 既存実装あり      │  │                              │  │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase C（Week 4）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ⑤ 物件資料・広告AI   │  │ ⑥ 内見・顧客管理（CRM）       │  │
│  │ マイソク+掲載文生成   │  │ 反響→マッチング→追客          │  │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase D（Week 5）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ⑦ 修繕・設備管理     │  │ ⑧ 免許・届出管理               │  │
│  │ 緊急度判定+長期修繕   │  │ 宅建業免許更新+報告書         │  │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  横断:                                                         │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 外部連携層（freee / 銀行API / ポータル連携 / 電子署名）  ||  │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 取引事例DB（国交省API + 路線価 + 自社成約実績）          ||  │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

---

# 1. 物件査定AI（★キラーフィーチャー）

## 1.1 問題の本質

```
不動産査定の現状:
  1. 物件の所在地・面積・築年数・構造等を確認する
  2. レインズ・ポータルで類似事例を探す（30分〜1時間）
  3. 取引事例比較法で概算を出す
  4. 収益還元法（投資物件の場合）を計算する
  5. 原価法で検算する
  6. 3手法を総合して査定価格を決定する
  7. 査定書を作成する（30分〜1時間）

  → 1件30分〜1時間。ベテランの勘に±20%のブレ
  → 査定ミスの損失: 安すぎれば売主流出、高すぎれば長期滞留→管理コスト増
  → 中小不動産会社では社長自身がやっている
```

## 1.2 査定AI — 3段階アプローチ

```
■ Level 1: 公開データ自動査定（Phase A）
  入力: 物件基本情報（所在地、面積、築年数、構造、間取り）
  処理:
    ① 国交省 不動産取引価格情報API で周辺取引事例を取得
    ② 国交省 地価公示・都道府県地価調査データで地価参照
    ③ 国税庁 路線価データで相続税評価額を算出
    ④ 3手法（取引事例比較法・収益還元法・原価法）で概算
    ⑤ LLMで物件固有の加減点を推定
  出力: 査定価格帯（下限〜上限）+ 査定書PDF
  精度目標: ±15%（取引事例が5件以上ある場合）

■ Level 2: 自社成約データ蓄積（Phase B）
  入力: Level 1 + 自社の成約実績
  処理:
    ① ユーザー入力のREINS事例（CSVアップロード or 手動入力）
    ② 自社成約データ（成約価格、媒介日数、値下げ経緯）
    ③ 成約確率シミュレーション
  出力: 「¥X万なら成約確率80%、¥X+200万なら50%」
  精度目標: ±8%

■ Level 3: ネットワーク効果（Phase 2+）
  入力: N社の匿名化成約データ
  処理: 地域×物件タイプ別の価格推定モデル
  出力: 市場動向レポート + 売り時/買い時アドバイス
```

## 1.3 査定計算ロジック

### 取引事例比較法（不動産鑑定評価基準 第7章）

```
査定価格 = Σ(事例価格 × 事情補正率 × 時点修正率 × 地域要因比較 × 個別要因比較) / 事例数

■ 事情補正率（特殊事情の排除）
  正常取引: 1.00
  売り急ぎ: 0.90〜0.95（理由: 市場価格より低く成約する傾向）
  買い進み: 1.05〜1.10（理由: 市場価格より高く成約する傾向）
  親族間取引: 0.80〜0.90
  競売: 0.60〜0.80
  ※ 補正率が0.85未満の事例は比較対象から除外

■ 時点修正率（取引時点→査定時点の市場変動）
  時点修正率 = 査定時点の地価指数 / 取引時点の地価指数

  地価指数ソース:
    ① 国交省 不動産価格指数（住宅）— 四半期更新
    ② 都道府県地価調査 — 毎年7/1基準
    ③ 地価公示 — 毎年1/1基準

  簡易計算:
    経過月数 = (査定年月 − 取引年月) の月数
    年間変動率 = 地域別の前年比変動率（地価公示データから取得）
    時点修正率 = 1 + (年間変動率 × 経過月数 / 12)

■ 地域要因比較（取引事例地域と対象地域の比較）
  地域要因比較 = 対象地域の標準的画地単価 / 事例地域の標準的画地単価

  主要比較要因と補正値:
    駅距離:
      徒歩5分以内:  +5%〜+10%
      徒歩10分:      ±0%（基準）
      徒歩15分超:   −5%〜−10%
      バス便:       −10%〜−20%
    用途地域:
      商業地域:     +10%〜+30%（容積率による）
      近隣商業:     +5%〜+15%
      第一種住居:    ±0%（基準）
      第一種低層:   −5%〜+5%（閑静な住宅地としての価値）
    前面道路:
      幅員6m以上:   +3%〜+5%
      幅員4m:        ±0%
      幅員4m未満:   −5%〜−15%（セットバック必要）
    嫌悪施設:
      なし:          ±0%
      あり:         −5%〜−20%（種類・距離による）

■ 個別要因比較（物件固有の条件）
  土地:
    形状: 整形+0%, 不整形−5%〜−20%, 旗竿地−10%〜−25%
    接道: 二方路+5%〜+10%, 角地+5%〜+15%, 袋地−20%〜−40%
```

```
    高低差: 道路面±0%, 高台+3%, 低地−5%〜−10%
    面積: 標準的画地±0%, 過小地（30坪未満）−5%〜−10%, 広大地−10%〜−30%
  建物（マンション・戸建の場合）:
    築年数: 経年減価率を適用（後述の原価法参照）
    階数（マンション）: 1階−3%〜−5%, 高層階+2%〜+5%/階, 最上階+5%〜+10%
    方位: 南向き±0%, 東向き−3%, 西向き−5%, 北向き−7%〜−10%
    リフォーム: 未実施±0%, 水回り+5%〜+10%, フルリノベ+10%〜+20%
```

## 収益還元法

```
■ 直接還元法（一時点の純収益で算定）
  査定価格 = 年間純収益（NOI） / 還元利回り（Cap Rate）

  年間純収益（NOI）:
    NOI = 潜在総収入（GPI） − 空室損失 − 運営費用（OPEX）

    潜在総収入（GPI）:
      = 月額賃料 × 12 + 共益費 × 12 + 駐車場収入 × 12 + 礼金・更新料の年間按分
      ※ レントロール（賃貸借条件一覧表）から取得

    空室損失:
      = GPI × 空室率
      空室率の推定:
        自社実績あり: 過去12ヶ月の実績空室率
        自社実績なし: 地域別平均空室率（LIFULL HOME'S空室率データ等）
        全国平均: 賃貸住宅20%前後（地域差大）

    運営費用（OPEX）:
      管理費:         GPI × 3%〜5%（管理会社への委託費）
      修繕積立金:     ¥200〜300/㎡/年（マンション）、¥150〜250/㎡/年（戸建賃貸）
      固定資産税:     固定資産税評価額 × 1.4%（標準税率）
      都市計画税:     固定資産税評価額 × 0.3%（市街化区域）
      火災保険料:     再調達価額 × 0.05%〜0.15%/年
      テナント募集費: 賃料1ヶ月分 × 想定回転率（2年で1回=0.5/年）

  還元利回り（Cap Rate）の決定:
    地域×物件タイプ別の参考値（2026年現在）:
    ┌──────────┬────────┬────────┬────────┬────────┐
    │ 物件タイプ │ 東京都心│ 東京23区│ 大阪市 │ 地方都市│
    ├──────────┼────────┼────────┼────────┼────────┤
    │ 区分マンション│ 3.5−4.5%│ 4.0−5.5%│ 4.5−6.0%│ 6.0−9.0%│
    │ 一棟AP     │ 4.0−5.5%│ 5.0−7.0%│ 5.5−7.5%│ 7.0−10.0%│
    │ 一棟マンション│ 3.5−5.0%│ 4.5−6.5%│ 5.0−7.0%│ 6.5−9.0%│
    │ 事務所     │ 3.0−4.5%│ 4.0−5.5%│ 4.5−6.5%│ 6.0−8.5%│
    │ 店舗      │ 3.5−5.0%│ 4.5−6.5%│ 5.0−7.5%│ 7.0−10.0%│
    └──────────┴────────┴────────┴────────┴────────┘

    Cap Rate補正:
      築浅（5年以内）: −0.3%〜−0.5%
      築古（30年超）:  +0.5%〜+1.5%
      駅近（5分以内）: −0.3%〜−0.5%
      駅遠（15分超）:  +0.3%〜+0.5%
      大規模修繕済み: −0.2%〜−0.3%

■ DCF法（Discounted Cash Flow − 将来CFの現在価値合計）
  査定価格 = Σ(NCFt / (1 + r)^t) + 復帰価格 / (1 + r)^n

    NCFt:   t年目のネットキャッシュフロー（NOI − 資本的支出）
    r:      割引率（Cap Rate + リスクプレミアム0.5%〜1.0%）
    n:      分析期間（通常5〜10年）
    復帰価格: n年目のNOI / ターミナルCap Rate
```

```
Phase 1では直接還元法のみ実装。DCFはPhase 2+。
```

## 原価法

```
■ 土地: 公示地価 or 路線価を基準に算出
  土地価格 = 路線価 × 面積 / 0.8（路線価は公示地価の80%水準）
  ※ 路線価がない場合: 固定資産税評価額 / 0.7 × 面積

■ 建物: 再調達原価法
  建物査定価格 = 再調達原価 × (1 − 経年減価率)

  再調達原価（新築想定の建築費）:
    ┌──────────────┬──────────┬──────────┐
    │ 構造          │ ㎡単価    │ 坪単価   │
    ├──────────────┼──────────┼──────────┤
    │ 木造（W）      │ 17−22万円 │ 56−73万円│
    │ 軽量鉄骨（LGS） │ 20−25万円 │ 66−83万円│
    │ 重量鉄骨（S）   │ 25−32万円 │ 83−106万円│
    │ RC造          │ 28−38万円 │ 93−126万円│
    │ SRC造         │ 32−42万円 │ 106−139万円│
    └──────────────┴──────────┴──────────┘

  経年減価率（法定耐用年数ベース + 実態補正）:
    法定耐用年数:
      木造:   22年
      軽量鉄骨: 27年（骨格材肉厚3mm超4mm以下）
      重量鉄骨: 34年（骨格材肉厚4mm超）
      RC造:   47年
      SRC造:  47年

    経年減価率の計算:
      築年数 ≤ 法定耐用年数の場合:
        減価率 = 築年数 / 法定耐用年数 × 0.9
        ※ 0.9は残存価値10%を考慮（税務上の残存は0だが実態は10%程度）

      築年数 > 法定耐用年数の場合:
        減価率 = 0.85 + min(0.10, (築年数 − 法定耐用年数) / 法定耐用年数 × 0.15)
        ※ 最大減価95%（最低でも5%の残存価値を認める）

    実態補正:
      大規模修繕済み:      減価率 × 0.85（15%回復）
      リフォーム済み（部分）: 減価率 × 0.90（10%回復）
      管理状態良好:        減価率 × 0.95（5%回復）
      管理状態不良:        減価率 × 1.05（5%追加減価）

■ 原価法査定価格 = 土地価格 + 建物査定価格
```

### 3手法の加重平均ロジック

```
最終査定価格 = W1 × 取引事例比較法 + W2 × 収益還元法 + W3 × 原価法

加重比率の決定（物件用途別）:
  ┌──────────────┬──────┬──────┬──────┐
  │ 物件用途       │ 事例法│ 収益法│ 原価法│
  ├──────────────┼──────┼──────┼──────┤
  │ 自用住宅（売買）│  0.50 │  0.10 │  0.40 │
  │ 投資用一棟     │  0.25 │  0.60 │  0.15 │
  │ 投資用区分     │  0.30 │  0.50 │  0.20 │
  │ 土地のみ       │  0.60 │  0.10 │  0.30 │
  │ 事業用（店舗等）│  0.20 │  0.60 │  0.20 │
  └──────────────┴──────┴──────┴──────┘

  ※ 事例数が3件未満の場合: 事例法の重みを半減し、他に按分
  ※ 収益データなし（自用住宅等）: 収益法の重みを0にし、事例法+原価法で按分

信頼度スコアの算出:
  confidence = min(
    事例数スコア（5件以上=1.0, 3件=0.8, 1件=0.5, 0件=0.2）,
    データ鮮度スコア（1年以内=1.0, 2年以内=0.8, 3年超=0.6）,
    地域カバー率（半径1km以内に事例あり=1.0, 3km=0.7, 5km超=0.4）,
  )
```

## 1.4 外部データソース

### 国交省 不動産取引価格情報API

```
■ エンドポイント
  GET https://www.land.mlit.go.jp/webland/api/TradeListSearch

■ パラメータ
  from:       取得開始期（例: 20231）= 2023年第1四半期
  to:         取得終了期（例: 20244）
  area:       都道府県コード（13=東京都）
  city:       市区町村コード（13101=千代田区）
  station:    最寄駅コード（省略可）
  TradePrice: 取引総額（省略可、フィルタ用）

■ レスポンス構造
  {
    "status": "OK",
    "data": [
      {
        "Type": "中古マンション等",
        "Region": "千代田区",
        "Municipality": "千代田区",
        "DistrictName": "麹町",
        "TradePrice": "45000000",          // 取引価格（円）
        "PricePerUnit": "",                // ㎡単価
        "FloorPlan": "2LDK",
        "Area": "65",                      // 面積㎡
        "UnitPrice": "",
        "LandShape": "",
        "Frontage": "",
        "TotalFloorArea": "",
        "BuildingYear": "2005",            // 築年
        "Structure": "ＲＣ",
        "Use": "住宅",
        "Purpose": "住宅",
        "Direction": "",
        "Classification": "",
        "Breadth": "",
        "CityPlanning": "商業地域",
        "CoverageRatio": "80",             // 建蔽率
        "FloorAreaRatio": "600",           // 容積率
        "Period": "2024年第1四半期",
        "Renovation": "",
        "Remarks": ""
      }
    ]
  }

■ 制約
  - 四半期ごとの集計データ（リアルタイムではない）
  - 個別物件を特定できない（プライバシー保護でノイズ付加あり）
```

- 1リクエストあたりの取得件数上限あり
- レート制限: 明文化されていないが、1秒1リクエスト程度を推奨

### 路線価データ

■ ソース: 国税庁 路線価図（全国）
https://www.rosenka.nta.go.jp/

■ 取得方法:
Phase 1: CSVアップロード方式（ユーザーが路線価図から読み取って入力）
Phase 2+: 路線価データ電子化サービスとのAPI連携

■ 路線価の補正（不整形地等）
路線価 × 奥行価格補正率 × 不整形地補正率 × 間口狭小補正率 × がけ地等補正率
※ 各補正率は国税庁「財産評価基準書」に基づく

## 1.5 データモデル

```
-- 査定プロジェクト
CREATE TABLE appraisal_projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  property_type TEXT NOT NULL,              -- land / house / apartment / mansion /
commercial / whole_building
  transaction_type TEXT NOT NULL,           -- sale / purchase / rent / investment
  address TEXT NOT NULL,                    -- 所在地
  prefecture TEXT NOT NULL,                 -- 都道府県
  municipality TEXT NOT NULL,               -- 市区町村
  district TEXT,                            -- 町丁目
  land_area DECIMAL(10,2),                 -- 土地面積（㎡）
  building_area DECIMAL(10,2),             -- 延床面積（㎡）
  building_year INTEGER,                    -- 築年（西暦）
  structure TEXT,                           -- W / LGS / S / RC / SRC
  floor_plan TEXT,                          -- 間取り（1LDK等）
  floor_number INTEGER,                     -- 所在階（マンション）
  total_floors INTEGER,                     -- 総階数
  direction TEXT,                           -- 主要開口方位（N/NE/E/SE/S/SW/W/NW）
  nearest_station TEXT,                     -- 最寄駅
  station_distance_min INTEGER,             -- 駅徒歩（分）
  zoning TEXT,                              -- 用途地域
  building_coverage_ratio DECIMAL(5,2),    -- 建蔽率（%）
  floor_area_ratio DECIMAL(5,2),           -- 容積率（%）
  road_width DECIMAL(4,1),                 -- 前面道路幅員（m）
  land_shape TEXT,                          -- 整形 / 不整形 / 旗竿地 / 三角地
  renovation_status TEXT,                   -- none / partial / full
  current_rent DECIMAL(12,2),              -- 現行賃料（投資物件の場合）
  vacancy_rate DECIMAL(5,2),               -- 空室率（%）
  appraised_price BIGINT,                  -- 査定価格（円）
  price_range_low BIGINT,                  -- 査定価格帯下限
  price_range_high BIGINT,                 -- 査定価格帯上限
  method_comparison BIGINT,                -- 取引事例比較法の算出額
  method_income BIGINT,                    -- 収益還元法の算出額
  method_cost BIGINT,                      -- 原価法の算出額
  confidence DECIMAL(3,2),                 -- 査定信頼度 0.00-1.00
  comparable_count INTEGER DEFAULT 0,       -- 使用した取引事例数
  status TEXT DEFAULT 'draft',              -- draft / appraised / proposed / contracted
  metadata JSONB DEFAULT '{}',              -- 補足条件等
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 取引事例（査定で使用した比較対象）
CREATE TABLE comparable_transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  appraisal_id UUID NOT NULL REFERENCES appraisal_projects(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  source TEXT NOT NULL,                     -- mlit_api / reins_manual / own_record / portal
  transaction_price BIGINT NOT NULL,        -- 取引価格（円）
  transaction_date DATE,                    -- 取引時期
```

```
  address TEXT,                              -- 所在地
  area DECIMAL(10,2),                       -- 面積（㎡）
  price_per_sqm DECIMAL(12,2),             -- ㎡単価
  situation_adjustment DECIMAL(5,3) DEFAULT 1.000, -- 事情補正率
  time_adjustment DECIMAL(5,3) DEFAULT 1.000,      -- 時点修正率
  area_adjustment DECIMAL(5,3) DEFAULT 1.000,      -- 地域要因補正
  individual_adjustment DECIMAL(5,3) DEFAULT 1.000, -- 個別要因補正
  adjusted_price BIGINT,                    -- 補正後価格
  raw_data JSONB DEFAULT '{}',              -- API生データ等
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 成約実績DB（自社蓄積。使うほど育つ）
CREATE TABLE transaction_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  property_type TEXT NOT NULL,
  address TEXT NOT NULL,
  prefecture TEXT NOT NULL,
  municipality TEXT NOT NULL,
  land_area DECIMAL(10,2),
  building_area DECIMAL(10,2),
  building_year INTEGER,
  structure TEXT,
  listed_price BIGINT,                      -- 売出価格
  contracted_price BIGINT,                  -- 成約価格
  price_reduction_rate DECIMAL(5,3),        -- 値下げ率
  days_on_market INTEGER,                   -- 媒介日数（売出→成約）
  contracted_date DATE,                     -- 成約日
  source TEXT,                               -- own / reins_manual
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE appraisal_projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE comparable_transactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE transaction_records ENABLE ROW LEVEL SECURITY;
```

## 1.6 パイプラインステップ

| Step | マイクロエージェント | 入力 | 処理 | 出力 |
|---|---|---|---|---|
| 1 | run_structured_extractor | property_data | 物件情報の取得・正規化（住所ゆらぎ吸収、築年数計算） | normalized_property |
| 2 | run_saas_reader | normalized_property | 外部データ自動収集（国交省API/路線価/地価公示/自社成約DB） | external_data |
| 3 | run_cost_calculator | external_data | 取引事例比較法の算出（事例選定→4段階補正→加重平均） | comparison_result |
| 4 | run_cost_calculator | comparison_result | 収益還元法の算出（NOI/Cap Rate。投資物件のみ） | income_result |
| 5 | run_cost_calculator | income_result | 原価法の算出（土地価格+建物再調達原価×経年減価率） | cost_result |
| 6 | run_rule_matcher | cost_result | 3手法の加重平均+信頼度スコア算出+価格帯算出 | synthesized_price |
| 7 | run_pdf_generator | synthesized_price | 査定書PDF生成（物件概要+算出過程+取引事例一覧+最終価格） | appraisal_report |

## 1.7 査定AIパイプライン（実装）

```
# workers/bpo/realestate/pipelines/appraisal_pipeline.py

class AppraisalPipeline:
    """
    物件査定AIパイプライン

    Step 1: property_reader     — 物件情報の取得・正規化
    Step 2: data_collector      — 外部データ自動収集（国交省API/路線価/地価）
    Step 3: comparison_method   — 取引事例比較法の算出
    Step 4: income_method       — 収益還元法の算出（投資物件のみ）
    Step 5: cost_method         — 原価法の算出
    Step 6: price_synthesizer   — 3手法の加重平均 + 信頼度算出
    Step 7: report_generator    — 査定書PDF生成
    """

    async def property_reader(self, input_data: dict) -> dict:
        """
        物件情報を取得・正規化する。
        - 住所の正規化（丁目・番地のゆらぎ吸収）
        - 都道府県コード・市区町村コードの付与
        - 築年数の自動計算（building_year → age_years）
        """

    async def data_collector(self, step1: dict) -> dict:
        """
        外部データを自動収集する。
        - 国交省 不動産取引価格情報API → 周辺取引事例リスト
        - 国交省 地価公示 → 基準地価
        - 路線価（Phase 1はユーザー入力、Phase 2でAPI化）
        - 自社 transaction_records → 成約実績
        """

    async def comparison_method(self, step2: dict) -> dict:
        """
        取引事例比較法で査定価格を算出する。
        - 事例選定（類似物件5〜10件を自動抽出）
        - 4段階補正（事情/時点/地域/個別）を各事例に適用
        - 補正後価格の加重平均（新しい事例ほど高ウェイト）
        """

    async def income_method(self, step3: dict) -> dict:
        """
        収益還元法（直接還元法）で査定価格を算出する。
        - transaction_type が investment / rent の場合のみ実行
        - NOI計算（GPI − 空室損 − OPEX）
        - Cap Rate推定（地域×物件タイプ×築年数から決定）
        - 査定価格 = NOI / Cap Rate
        """

    async def cost_method(self, step4: dict) -> dict:
        """
```

```
        原価法で査定価格を算出する。
        - 土地: 路線価 or 地価公示ベース
        - 建物: 再調達原価 × (1 - 経年減価率)
        - 経年減価率の実態補正（修繕/管理状態）
        """

    async def price_synthesizer(self, step5: dict) -> dict:
        """
        3手法の加重平均と信頼度スコアを算出する。
        - 物件用途別の加重比率テーブル参照
        - 各手法のデータ品質による動的ウェイト調整
        - confidence算出（事例数×鮮度×地域カバー率）
        - 価格帯（±信頼区間）の算出
        """

    async def report_generator(self, step6: dict) -> AppraisalResult:
        """
        査定書PDFを生成する。
        - 物件概要
        - 3手法の算出過程
        - 使用した取引事例一覧（地図プロット付き）
        - 最終査定価格と根拠
        - 成約確率シミュレーション（Level 2+）
        """
```

## 1.7 法令参照

■ 不動産鑑定評価基準（国交省）
- 第7章 鑑定評価の方式（取引事例比較法、収益還元法、原価法）
- 別表 各種補正率

■ 不動産の鑑定評価に関する法律
- 第3条: 不動産鑑定業の登録

※ AIによる査定は「鑑定評価」ではなく「価格査定」の位置づけ。
宅建業者が媒介の参考として行う査定は鑑定評価に該当しない（判例・通説）

■ 宅地建物取引業法
- 第34条の2: 媒介契約（専任媒介の場合、価額の意見の根拠を書面で示す義務）
- 第34条の2 第2項: 意見の根拠の明示義務
- 第35条: 重要事項の説明（買主/借主に対する説明義務）
- 第37条: 書面の交付（契約書面の交付義務）
- 第40条: 瑕疵担保責任についての特約の制限
  （売主が宅建業者の場合、引渡しから2年以上の期間でなければ特約無効）
- 第47条: 業務に関する禁止事項（重要事実の不告知、不実告知の禁止）

■ 借地借家法
- 第26条: 建物賃貸借の更新（法定更新の規定）
- 第28条: 建物賃貸借契約の更新拒絶の要件（正当事由が必要）
- 第38条: 定期建物賃貸借
- 第40条: 一時使用目的の建物の賃貸借

■ 民法（2020年改正後）
- 第562–564条: 売買の契約不適合責任
- 第601条〜第622条: 賃貸借（定義、修繕義務、原状回復、敷金）
- 特に第621条（賃借人の原状回復義務）、第622条の2（敷金）

■ 建物の区分所有等に関する法律（区分所有法/マンション管理法）
- 第3条: 区分所有者の団体（管理組合の当然成立）
- 第17条: 共用部分の変更（特別多数決議 = 3/4以上）
- 第30条: 規約事項

※ マンション管理の重要事項説明時に管理組合の決議事項チェックが必要

■ 消費者契約法
- 第9条: 損害賠償額の予定条項の制限
- 第10条: 消費者の利益を一方的に害する条項の無効

■ 国税庁 財産評価基準書
- 路線価方式による評価
- 各種補正率（奥行価格、不整形地、間口狭小等）

# 2. 契約書AI自動生成

## 2.1 問題の本質

```
不動産契約書の現状:
  - 1件あたり1-3時間の作成時間
  - 宅建業法35条（重要事項説明書）の記載漏れリスク
  - 宅建業法37条（契約書面）の必須事項チェック漏れ
  - 特約条項の策定が属人的（ベテラン営業の知見に依存）
  - 法改正への追従が遅れがち
  - 印紙税の節約ニーズ（電子契約化）

  → 記載漏れは行政処分（業務停止命令）のリスク
  → 電子契約化で印紙税削減（売買契約1件あたり¥1-6万の節約）
```

## 2.2 宅建業法 35条 重要事項説明書の必須記載事項

```
■ 物件に関する事項（宅建業法35条1項各号）
  1号:  登記された権利の種類・内容
  2号:  法令上の制限（用途地域、建蔽率、容積率、防火指定等）
  3号:  私道負担（負担面積・負担金）
  4号:  飲用水・電気・ガスの供給施設、排水施設の整備状況
  5号:  未完成物件の場合の完了時の形状・構造
  6号:  建物状況調査（インスペクション）の有無・結果
  6号の2: 建物のエネルギー消費性能（省エネ性能説明義務: 2025年4月〜）
  7号:  代金・交換差金以外に授受される金銭の額・目的
  8号:  契約の解除に関する事項
  9号:  損害賠償額の予定・違約金
  10号: 手付金等の保全措置（売主が宅建業者の場合）
  11号: 支払金・預り金の保全措置
  12号: 金銭の貸借のあっせん（ローン特約）
  13号: 契約不適合責任（瑕疵担保）の定め
  14号: 供託所等に関する事項

■ 区分所有建物（マンション）追加事項（35条1項各号続き）
  - 専有部分の用途制限
  - 管理費・修繕積立金
  - 管理の委託先
  - 建物の維持修繕の実施状況
  - 管理組合の決議事項

■ 賃貸特有の記載事項
  - 台所、浴室、便所等の設備の整備状況
  - 敷金等の精算に関する事項（2020年民法改正対応: 原状回復ガイドライン）
  - 定期建物賃貸借の場合はその旨（借地借家法38条）

■ IT重説対応（2021年〜全面解禁）
  - オンライン重説の要件チェック（映像・音声の確認、書面の事前送付等）
```

## 2.3 宅建業法 37条 契約書面の必須記載事項

■ 売買・交換の場合（37条1項各号）
1号:　当事者の氏名・住所
2号:　物件を特定するために必要な表示
3号:　代金・交換差金の額、支払時期・方法
4号:　物件の引渡しの時期
5号:　移転登記の申請の時期
6号:　代金以外の金銭の額、授受の時期・目的（手付金、固定資産税精算金等）
7号:　契約の解除に関する定め
8号:　損害賠償額の予定・違約金
9号:　天災等による損害の負担（危険負担）
10号: 契約不適合責任（種類・品質に関する）
11号: 公租公課の負担
12号: ローン特約

■ 賃貸の場合（37条2項各号）
1号:　当事者の氏名・住所
2号:　物件の表示
3号:　賃料の額、支払時期・方法
4号:　物件の引渡しの時期
5号:　借賃以外の金銭の額、授受の時期・目的（敷金、礼金、保証金等）
6号:　契約の解除に関する定め
7号:　損害賠償額の予定・違約金

## 2.4 特約条項の自動生成ロジック

```
■ LLM生成 + ルールベースチェックのハイブリッド

Step 1: 取引条件から必要な特約を分類
  条件マッピング:
    物件が築30年超            → 契約不適合責任免責特約
    売主が宅建業者            → 2年以上の契約不適合責任義務（宅建業法40条）
    ローン利用あり            → ローン特約（白紙解除条項）
    引渡前のリフォームあり    → リフォーム条件特約
    借地権付き建物            → 借地権承諾特約
    定期借家契約              → 定期借家の特約事項（借地借家法38条）
    ペット可賃貸              → ペット飼育特約（種類・数・原状回復）
    事業用賃貸                → 使用目的制限特約
    テナント退去予定          → 引渡条件特約（空渡し/居抜き）

Step 2: LLMが物件・取引固有の特約文面を生成
  プロンプト:
    「以下の取引条件に基づき、売買（賃貸借）契約の特約条項を生成してください。
     各条項は宅建業法・民法・借地借家法に準拠し、
     売主（貸主）・買主（借主）双方にとって公平な内容としてください。」

Step 3: ルールベースの法令違反チェック
  禁止パターン:
    × 売主が宅建業者で契約不適合責任を完全免除 → 宅建業法40条違反
    × 消費者契約で一方的に不利な違約金条項 → 消費者契約法9条・10条
    × 更新拒絶を無条件に認める条項 → 借地借家法28条（正当事由必要）
    × 敷金全額返還しない条項 → 民法621条・622条の2に抵触の可能性
    × 原状回復で通常損耗を借主負担とする条項 → 判例（最判H17.12.16）
```

## 2.5 データモデル

```
-- 契約書プロジェクト
CREATE TABLE contract_projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  contract_type TEXT NOT NULL,              -- sale / purchase / lease / sublease /
management
  property_id UUID,                         -- 関連物件ID（appraisal_projects等と連携可能）
  property_address TEXT NOT NULL,
  parties JSONB NOT NULL,                   -- {seller: {name, address}, buyer: {name,
address}} or {landlord, tenant}
  terms JSONB NOT NULL,                     -- {price, payment_schedule, delivery_date, ...}
  special_conditions JSONB DEFAULT '[]',    -- [{title, body, law_reference}]
  important_explanation JSONB DEFAULT '{}', -- 35条書面データ
  contract_document JSONB DEFAULT '{}',     -- 37条書面データ
  risk_check_result JSONB DEFAULT '[]',     -- [{severity, message, law_reference}]
  generated_files JSONB DEFAULT '[]',       -- [{type: "35_doc", url: "...", version: 1}]
  status TEXT DEFAULT 'draft',              -- draft / review / approved / signed / cancelled
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 契約書テンプレート
CREATE TABLE contract_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  template_type TEXT NOT NULL,              -- sale_residential / sale_commercial /
lease_residential / lease_commercial / management
  name TEXT NOT NULL,
  clauses JSONB NOT NULL,                   -- [{section, title, body, is_required,
law_reference}]
  special_conditions_templates JSONB DEFAULT '[]', -- [{condition_key, title, body_template}]
  version INTEGER DEFAULT 1,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE contract_projects ENABLE ROW LEVEL SECURITY;
-- contract_templates は全社共通（RLS不要、読み取り専用）
```

## 2.6 パイプラインステップ

| Step | マイクロエージェント | 入力 | 処理 | 出力 |
|---|---|---|---|---|
| 1 | run_structured_extractor | terms_input | 取引条件の取得・正規化、必須項目欠損チェック | normalized_terms |
| 2 | run_rule_matcher | normalized_terms | contract_typeに基づくテンプレート選択・基本条項設定 | template_with_terms |
| 3 | run_document_generator | template_with_terms | 35条/37条の必須記載事項を取引条件から自動埋め | clauses_filled |
| 4 | run_document_generator | clauses_filled | 特約条項のLLM生成（条件マッピング→テンプレ+LLM補完） | with_special_conditions |
| 5 | run_compliance_checker | with_special_conditions | 全条項の法令違反チェック（宅建業法40条/消費者契約法/借地借家法） | risk_checked |
| 6 | run_pdf_generator | risk_checked | Word/PDF出力+電子署名連携用メタデータ付与 | contract_document |

## 2.7 契約書AIパイプライン（実装）

```
# workers/bpo/realestate/pipelines/contract_pipeline.py

class ContractPipeline:
    """
    契約書AI自動生成パイプライン

    Step 1: terms_reader         — 取引条件の取得・正規化
    Step 2: template_selector    — テンプレート選択・基本条項設定
    Step 3: clause_generator     — 35条/37条の必須記載事項を自動埋め
    Step 4: special_conditions   — 特約条項のLLM生成
    Step 5: risk_checker         — 法令違反・リスクチェック
    Step 6: document_generator   — Word/PDF出力
    """

    async def terms_reader(self, input_data: dict) -> dict:
        """取引条件を正規化。必須項目の欠損チェック。"""

    async def template_selector(self, step1: dict) -> dict:
        """contract_type に基づいてテンプレートを選択。"""

    async def clause_generator(self, step2: dict) -> dict:
        """
        35条/37条の必須記載事項を取引条件から自動埋め。
        - 法令上の制限は住所から用途地域等を参照
        - 設備状況は物件データから自動取得
        - 不足情報はplaceholder + 手動入力依頼
        """

    async def special_conditions(self, step3: dict) -> dict:
        """
        LLMで特約条項を生成。条件マッピング→テンプレ+LLM補完。
        借地借家法38条（定期借家）等の必要な特約を自動判定。
        """

    async def risk_checker(self, step4: dict) -> dict:
        """
        全条項の法令違反チェック。
        - 宅建業法40条（瑕疵担保の制限）
        - 消費者契約法9条・10条
        - 借地借家法28条（正当事由）
        - 民法621条（原状回復）
        severity: error（法令違反）/ warning（リスクあり）/ info（推奨）
        """

    async def document_generator(self, step5: dict) -> ContractResult:
        """Word/PDF生成。電子署名連携用のメタデータ付与。"""
```

### 2.7 法令参照

```
■ 宅地建物取引業法
  - 35条: 重要事項の説明
  - 37条: 書面の交付
  - 40条: 瑕疵担保責任についての特約の制限
  - 47条: 業務に関する禁止事項（重要事実の不告知等）

■ 民法（2020年改正後）
  - 562-564条: 売買の契約不適合責任
  - 601条: 賃貸借
  - 621条: 賃借人の原状回復義務
  - 622条の2: 敷金

■ 借地借家法
  - 26条: 建物賃貸借の更新（法定更新）
  - 28条: 建物賃貸借契約の更新拒絶の要件（正当事由）
  - 38条: 定期建物賃貸借
  - 40条: 一時使用目的の建物の賃貸借

■ 消費者契約法
  - 9条: 消費者が支払う損害賠償の額を予定する条項等の無効
  - 10条: 消費者の利益を一方的に害する条項の無効
```

---

## 3. 賃料・収支管理（✅ 既存実装: rent_collection_pipeline.py）

### 3.1 既存実装の概要

```
実装済み: workers/bpo/realestate/pipelines/rent_collection_pipeline.py
  Step 1: tenant_reader      — 入居者・家賃データ取得
  Step 2: payment_checker    — 入金確認
  Step 3: arrears_calculator — 滞納計算（遅延損害金: 年6%）
  Step 4: notice_drafter     — 催告書・督促状ドラフト生成（LLM）
  Step 5: output_validator   — バリデーション
```

## 3.2 パイプラインステップ

| Step | マイクロエージェント | 入力 | 処理 | 出力 |
|---|---|---|---|---|
| 1 | run_saas_reader | property_id, period | 入居者・家賃データ取得 | tenant_rent_data |
| 2 | run_rule_matcher | tenant_rent_data + bank_data | 入金確認（名義ファジーマッチ含む） | payment_status |
| 3 | run_cost_calculator | payment_status | 滞納計算（遅延損害金: 年率6%/契約未記載は3%） | arrears_result |
| 4 | run_message_drafter | arrears_result | 催告書・督促状ドラフト生成（LLM。4段階フロー） | notice_draft |
| 5 | run_output_validator | notice_draft | バリデーション（法定記載事項チェック） | validated_output |

## 3.3 追加実装仕様

### 入金消込の名義ゆらぎ対応

```
■ 口座振込の名義ゆらぎパターン
  正式名: 「山田太郎」
  ゆらぎ例:
    - ヤマダ タロウ（カタカナ全角）
    - ﾔﾏﾀﾞ ﾀﾛｳ（カタカナ半角）
    - ヤマダタロウ（スペースなし）
    - YAMADA TARO（ローマ字）
    - 山田太朗（同音異字）
    - 山田（姓のみ）
    - ヤマダ（姓のみカナ）
    - 山田太郎（株（会社名義）

■ マッチングアルゴリズム（3段階）
  Stage 1: 完全一致チェック
    口座名義 == テナント名 or テナント名カナ → 確定（confidence=1.0）

  Stage 2: ファジーマッチング
    - カタカナ正規化（半角→全角、濁点統一）
    - スペース除去
    - 姓のみマッチ + 部屋番号 + 金額一致 → 候補（confidence=0.8）
    - レーベンシュタイン距離 ≤ 2 → 候補（confidence=0.7）

  Stage 3: LLMマッチング（Stage 2で候補が複数 or confidence < 0.7の場合）
    - 口座名義 + 入金額 + 入金日 → テナントリストから最適マッチを推定
    - confidence=0.5〜0.8（LLM推定結果による）

  ※ confidence < 0.7 は手動確認フラグを立てる
```

## 滞納督促4段階フロー

```
■ 督促フロー（既存実装を拡張）

  Stage 1: 自動SMS/LINE通知（滞納1日後）
    トリガー: payment_due_date + 1日を経過
    チャネル: SMS（LINE連携済みの場合はLINE優先）
    文面: 「【{物件名}】{月}月分の家賃{金額}円のお支払いが確認できておりません。
            お忘れの場合は至急のお振込みをお願いいたします。」
    制約: 夜間（21:00–8:00）は送信しない（貸金業法21条準拠、賃貸にも準用）

  Stage 2: 電話連絡（滞納3日後）
    トリガー: Stage 1 実施後 + 入金未確認
    アクション: 電話タスクを担当者に自動割り当て（CRM連携）
    通話メモテンプレート自動生成
    ※ 自動発信はPhase 2+。Phase 1はタスク生成のみ

  Stage 3: 書面催告（滞納7日後）
    トリガー: Stage 2 実施後 + 入金未確認
    文書: 催告書（内容証明ではない普通郵便）
    LLM生成 → 担当者確認 → 自動発送（クラウドレター連携 Phase 2+）

  Stage 4: 内容証明郵便（滞納14日後）
    トリガー: Stage 3 実施後 + 入金未確認
    文書: 内容証明郵便テンプレート
    内容:
      - 未払賃料の金額と期間
      - 遅延損害金の計算明細
      - 支払期限（到達後7日以内）
      - 「期限内に支払いなき場合、契約解除及び明渡請求の法的手続きに移行」
    ※ 内容証明の発送は手動（eKYC済みのe内容証明連携はPhase 2+）
    法令: 民法541条（催告解除）、借地借家法28条不適用（賃借人の債務不履行）

■ 遅延損害金計算
  計算式: 滞納額 × 年率 / 365 × 滞納日数
  年率:
    契約に定めあり: 契約記載の率（上限: 年14.6%）
    契約に定めなし: 法定利率 年3%（民法404条、2023年4月〜）
    ※ 既存実装は年6%（旧商事法定利率）→ 契約に定めがある前提。
       契約未記載の場合は年3%にフォールバックする実装を追加する。

  計算例:
    家賃: ¥80,000、滞納30日、年率6%の場合
    遅延損害金 = 80,000 × 0.06 / 365 × 30 = ¥394
```

## 3.3 オーナー送金・収支レポート

```
■ オーナー送金明細の自動生成

  送金額 = 月額賃料収入
           − 管理手数料（賃料 × 管理委託料率 3−5%）
           − 当月修繕費（オーナー負担分）
           − 広告料按分（テナント募集費の按分）
           + 更新料分配（更新料のオーナー取り分: 通常50%）
           + 滞納回収分（前月以前の回収済み滞納賃料）

  出力: PDF明細書（freee連携で仕訳データ自動連携 Phase 2+）

■ 収支レポート
  物件別月次レポート:
    − 賃料収入（実績 vs 潜在）
    − 空室率推移（過去12ヶ月グラフ）
    − 管理費・修繕費推移
    − NOI（ネット営業収益）推移
    − 滞納状況サマリー

  オーナー別年間レポート:
    − 確定申告用 不動産所得計算書の下書き
    − 減価償却計算の自動化
    − 必要経費の集計（管理費、修繕費、保険料、固定資産税等）
```

## 3.4 データモデル（追加分）

```
-- 管理物件マスタ
CREATE TABLE managed_properties (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  property_name TEXT NOT NULL,              -- 物件名
  address TEXT NOT NULL,                    -- 所在地
  owner_name TEXT NOT NULL,                -- オーナー名
  owner_id UUID,                           -- オーナーID（顧客マスタ連携）
  total_units INTEGER NOT NULL,             -- 総戸数
  management_fee_rate DECIMAL(5,3),        -- 管理委託料率（例: 0.05 = 5%）
  management_start_date DATE,              -- 管理開始日
  property_type TEXT,                       -- apartment / mansion / house / commercial
  structure TEXT,                           -- W / S / RC / SRC
  building_year INTEGER,                    -- 築年
  status TEXT DEFAULT 'active',             -- active / terminated
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 入居者マスタ
CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  property_id UUID NOT NULL REFERENCES managed_properties(id),
  room_number TEXT NOT NULL,                -- 部屋番号
  tenant_name TEXT NOT NULL,                -- 入居者名
  tenant_name_kana TEXT,                    -- 入居者名カナ
  phone TEXT,
  email TEXT,
  line_id TEXT,                             -- LINE連携用
  monthly_rent INTEGER NOT NULL,            -- 月額賃料
  common_fee INTEGER DEFAULT 0,             -- 共益費
  deposit INTEGER DEFAULT 0,                -- 敷金（預かり額）
  lease_start_date DATE NOT NULL,           -- 契約開始日
  lease_end_date DATE,                      -- 契約終了日
  lease_type TEXT DEFAULT 'ordinary',       -- ordinary（普通借家）/ fixed_term（定期借家）
  payment_due_day INTEGER DEFAULT 27,       -- 毎月の支払期日
  late_fee_rate DECIMAL(5,3) DEFAULT 0.060, -- 遅延損害金年率
  bank_account_name TEXT,                   -- 振込人名義（消込用）
  status TEXT DEFAULT 'active',             -- active / notice_given / vacated
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 入金記録
CREATE TABLE payment_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  tenant_id UUID REFERENCES tenants(id),
```

```
  property_id UUID NOT NULL REFERENCES managed_properties(id),
  payment_date DATE NOT NULL,               -- 入金日
  amount INTEGER NOT NULL,                  -- 入金額
  payment_type TEXT DEFAULT 'rent',         -- rent / deposit / key_money / renewal / other
  period_year INTEGER,                      -- 対象年
  period_month INTEGER,                     -- 対象月
  bank_transfer_name TEXT,                  -- 振込人名義（生データ）
  match_confidence DECIMAL(3,2),           -- 消込マッチ信頼度
  match_method TEXT,                        -- exact / fuzzy / llm / manual
  status TEXT DEFAULT 'matched',            -- matched / unmatched / manual_review
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 督促履歴
CREATE TABLE collection_actions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  action_stage INTEGER NOT NULL,            -- 1=SMS, 2=電話, 3=書面, 4=内容証明
  action_date DATE NOT NULL,
  action_detail TEXT,                       -- 実施内容メモ
  result TEXT,                              -- sent / reached / no_answer / paid / escalated
  document_url TEXT,                        -- 生成した文書のURL
  created_at TIMESTAMPTZ DEFAULT now()
);

-- オーナー送金記録
CREATE TABLE owner_remittances (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  property_id UUID NOT NULL REFERENCES managed_properties(id),
  period_year INTEGER NOT NULL,
  period_month INTEGER NOT NULL,
  total_rent_collected INTEGER NOT NULL,    -- 当月回収賃料合計
  management_fee INTEGER NOT NULL,          -- 管理手数料
  repair_cost INTEGER DEFAULT 0,            -- 修繕費（オーナー負担分）
  ad_cost INTEGER DEFAULT 0,               -- 広告料
  renewal_fee_share INTEGER DEFAULT 0,     -- 更新料オーナー分配
  arrears_collected INTEGER DEFAULT 0,     -- 滞納回収分
  remittance_amount INTEGER NOT NULL,       -- 送金額
  remittance_date DATE,                     -- 送金日
  statement_url TEXT,                       -- 送金明細PDF URL
  status TEXT DEFAULT 'calculated',         -- calculated / approved / remitted
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE managed_properties ENABLE ROW LEVEL SECURITY;
ALTER TABLE tenants ENABLE ROW LEVEL SECURITY;
ALTER TABLE payment_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE collection_actions ENABLE ROW LEVEL SECURITY;
ALTER TABLE owner_remittances ENABLE ROW LEVEL SECURITY;
```

# 4. 送金・入金管理

## 4.1 敷金台帳管理ロジック

```
■ 敷金のライフサイクル

  預かり（入居時）
    → 敷金台帳に記録（tenant_id, amount, received_date）
    → 預り金として負債計上

  保管中
    → 敷金台帳で残高管理
    → オーナー別の預り敷金合計を月次レポート

  精算（退去時）
    精算額 = 敷金預り額 − 原状回復費用（借主負担分）

    原状回復費用の算定ルール（国交省「原状回復をめぐるトラブルとガイドライン」）:
      ■ 貸主負担（通常損耗・経年劣化）
        − 壁紙の日焼け、画鋲の穴
        − フローリングの日焼け
        − 畳の日焼け・表替え
        − 設備の経年劣化（耐用年数経過後）
      ■ 借主負担（故意・過失・善管注意義務違反）
        − タバコのヤニ汚れ
        − ペットによる傷・汚損
        − 釘穴・ネジ穴（下地ボード損傷）
        − 借主の不注意による水漏れ跡

    経過年数考慮（借主負担の減額）:
      残存価値 = 1 − (入居年数 / 耐用年数)
      借主負担額 = 修繕費 × max(残存価値, 0.10)

      耐用年数の目安:
        壁紙（クロス）: 6年
        カーペット:     6年
        フローリング:   耐用年数なし（部分補修は経過年数考慮なし）
        設備機器:       設備ごとの法定耐用年数

  返還
    返還額 = 敷金 − 借主負担の原状回復費
    返還期限: 退去後1ヶ月以内（民法622条の2 第1項 − 明渡し時に返還義務発生）

■ 敷金台帳データモデル
  CREATE TABLE deposit_ledger (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    property_id UUID NOT NULL REFERENCES managed_properties(id),
    deposit_amount INTEGER NOT NULL,          -- 預り敷金額
    received_date DATE NOT NULL,              -- 預り日
    restoration_cost INTEGER DEFAULT 0,       -- 原状回復費（借主負担分）
```

```
  restoration_detail JSONB DEFAULT '[]',    -- [{item, cost, tenant_share_rate, reason}]
  refund_amount INTEGER,                    -- 返還額
  refund_date DATE,                         -- 返還日
  status TEXT DEFAULT 'held',               -- held / calculating / refunded
  created_at TIMESTAMPTZ DEFAULT now()
);
ALTER TABLE deposit_ledger ENABLE ROW LEVEL SECURITY;
```

## 4.2 インボイス制度対応

```
■ 適格請求書の必須記載事項（消費税法57条の4）

  1. 適格請求書発行事業者の氏名又は名称及び登録番号（T+13桁）
  2. 取引年月日
  3. 取引の内容（軽減税率の対象品目である旨）
  4. 税率ごとに区分した対価の額（税抜又は税込）及び適用税率
  5. 税率ごとに区分した消費税額等
  6. 書類の交付を受ける事業者の氏名又は名称

■ 不動産業における消費税の注意点
  非課税取引:
    - 住宅の貸付け（居住用賃貸: 消費税非課税）
    - 土地の譲渡・貸付け
  課税取引:
    - 事業用建物の貸付け
    - 建物の譲渡
    - 仲介手数料
    - 管理委託料
    - 駐車場の貸付け（施設を設置している場合）

■ 仲介手数料の上限チェック（宅建業法46条 + 報酬告示）
  売買の場合:
    取引価格200万円以下:        代金 × 5.5%（税込）
    200万円超〜400万円以下:     代金 × 4.4%（税込）
    400万円超:                  代金 × 3.3%（税込）
    速算式: 取引価格 × 3.3% + 66,000円（税込）※400万超の場合

  賃貸の場合:
    上限: 賃料の1.1ヶ月分（税込）
    ※ 依頼者の一方から受領できるのは0.55ヶ月分が原則
      （依頼者の承諾がある場合は1.1ヶ月分まで可）

  自動チェック:
    請求書生成時に仲介手数料が上限を超えていないかバリデーション。
    超過の場合は error で生成を停止。
```

## 4.3 パイプラインステップ

| Step | マイクロエージェント | 入力 | 処理 | 出力 |
|---|---|---|---|---|
| 1 | run_saas_reader | company_id, period | 取引データ取得（入金記録・契約データ・敷金台帳） | transaction_data |
| 2 | run_document_generator | transaction_data | 適格請求書生成（インボイス制度対応。T+13桁、税率区分） | invoice_draft |
| 3 | run_compliance_checker | invoice_draft | 仲介手数料上限チェック（宅建業法46条。超過時はerrorで生成停止） | validated_invoice |
| 4 | run_rule_matcher | validated_invoice | 敷金台帳更新（退去時精算=敷金-原状回復費。経過年数考慮） | deposit_updated |
| 5 | run_cost_calculator | deposit_updated | オーナー送金額計算（賃料-管理手数料-修繕費+更新料分配） | remittance_calc |
| 6 | run_pdf_generator | remittance_calc | 帳票出力（請求書PDF+送金明細PDF） | output_documents |

## 4.4 パイプライン（実装）

```
# workers/bpo/realestate/pipelines/billing_pipeline.py

class BillingPipeline:
    """
    送金・入金管理パイプライン

    Step 1: transaction_reader   — 取引データ取得
    Step 2: invoice_generator    — 適格請求書生成（インボイス対応）
    Step 3: fee_validator        — 仲介手数料上限チェック
    Step 4: deposit_manager      — 敷金台帳更新
    Step 5: remittance_calc      — オーナー送金額計算
    Step 6: output_generator     — 帳票出力
    """
```

# 5. 物件資料・広告作成AI

## 5.1 マイソク自動生成

```
■ マイソク（物件概要書/販売図面/募集図面）の構成

  ┌──────────────────────────────────────┐
  │ ヘッダー: 取引態様（売主/仲介）、物件種別    │
  ├──────────────────┬───────────────────┤
  │                  │                   │
  │  間取り図          │  物件写真（メイン）      │
  │   （配置図面）      │                   │
  │                  │                   │
  ├──────────────────┴───────────────────┤
  │ 物件概要                                 │
  │  所在地 / 交通 / 土地面積 / 建物面積        │
  │  構造 / 築年月 / 間取り / 階数              │
  │  用途地域 / 建蔽率・容積率                  │
  │  設備（バス・トイレ別、エアコン、駐車場等）  │
  │  価格 or 賃料（管理費・共益費含む）         │
  ├──────────────────────────────────────┤
  │ アクセス情報（最寄駅、バス停、車移動時間）   │
  ├──────────────────────────────────────┤
  │ 備考・特記事項                            │
  ├──────────────────────────────────────┤
  │ 取引条件（仲介手数料、引渡時期等）          │
  ├──────────────────────────────────────┤
  │ 会社情報（社名、免許番号、担当者、連絡先）  │
  └──────────────────────────────────────┘

■ 自動生成ロジック
  Step 1: 物件データベースから基本情報を取得
  Step 2: 間取り図画像を配置（アップロード済み or 自動生成 Phase 2+）
  Step 3: 物件写真を自動選定（Vision AIでメイン写真を選定）
  Step 4: LLMでキャッチコピー・セールスポイントを生成
  Step 5: PDF/画像に自動レイアウト（テンプレートエンジン）

■ 不動産広告の法令制約（不動産の表示に関する公正競争規約）
  禁止用語:
    × 「最高」「最上級」「格安」「掘出し」「完璧」
    × 「日本一」「業界No.1」（根拠のない比較表現）
    × 「徒歩0分」（1分未満でも「徒歩1分」と表示）
  必須表示:
    ✓ 徒歩所要時間: 80m = 1分（端数切り上げ）
    ✓ 交通利便: 最寄駅からの実際の道路距離
    ✓ 建築年月: 築年数ではなく建築年月を表示
    ✓ 取引態様: 売主 / 代理 / 仲介（媒介）
    ✓ 免許番号
  LLMチェック:
    生成されたテキストに禁止用語が含まれていないかバリデーション
```

## 5.2 ポータル掲載文LLM生成

```
■ 対応ポータル
  - SUUMO（リクルート）
  - HOME'S（LIFULL）
  - at home（アットホーム）
  - Yahoo!不動産

■ LLMプロンプト
  「以下の物件情報に基づき、不動産ポータルサイトの掲載文を作成してください。
   【制約】
   - 不動産公正競争規約に準拠すること
   - 禁止用語（最高、格安、掘出し等）を使用しないこと
   - 物件の具体的な魅力を3つ以上挙げること
   - 想定する入居者/購入者のペルソナに訴求する文体にすること
   - 300文字以内で簡潔にまとめること
   【物件情報】
  {property_data}」

■ ABテスト機能（Phase 2+）
  - 2パターンの掲載文を生成
  - 反響率（問合せ数/PV数）を計測
  - 高パフォーマンス文面を自動採用
```

## 5.3 パイプラインステップ

| Step | マイクロエージェント | 入力 | 処理 | 出力 |
|---|---|---|---|---|
| 1 | run_structured_extractor | property_id | 物件情報取得・正規化 | normalized_property |
| 2 | run_document_ocr | photo_files | 写真分類・メイン写真選定（Vision AI） | classified_photos |
| 3 | run_document_generator | normalized_property + classified_photos | LLMでキャッチコピー・掲載文生成 | ad_copy |
| 4 | run_compliance_checker | ad_copy | 公正競争規約チェック（禁止用語・必須表示バリデーション） | compliant_copy |
| 5 | run_pdf_generator | compliant_copy + classified_photos | マイソクPDF生成（テンプレートエンジン） | maisoku_pdf |
| 6 | run_document_generator | compliant_copy | ポータルサイト別CSV/データ生成（SUUMO/HOME'S/at home/Yahoo!） | portal_data |
| 7 | run_output_validator | maisoku_pdf + portal_data | バリデーション（記載事項完全性チェック） | validated_output |

## 5.4 パイプライン（実装）

```
# workers/bpo/realestate/pipelines/listing_pipeline.py

class ListingPipeline:
    """
    物件資料・広告作成AIパイプライン

    Step 1: property_reader      — 物件情報取得・正規化
    Step 2: photo_processor      — 写真分類・メイン写真選定
    Step 3: copy_generator       — LLMでキャッチコピー・掲載文生成
    Step 4: compliance_checker   — 公正競争規約チェック（禁止用語等）
    Step 5: maisoku_generator    — マイソクPDF生成
    Step 6: portal_formatter     — ポータルサイト別CSV/データ生成
    Step 7: output_validator     — バリデーション
    """
```

---

# 6. 内見・顧客管理（CRM）

## 6.1 反響→マッチング→追客の自動化フロー

```
■ 反響受付の自動化

  流入チャネル:
    ① ポータルサイト反響（SUUMO/HOME'S等 → メール自動取込）
    ② 自社HP問合せフォーム
    ③ LINE公式アカウント
    ④ 電話（STT→テキスト化 Phase 2+）
    ⑤ 来店

  自動処理:
    反響メール受信
      → メール本文をLLMで構造化
        - 顧客名、連絡先、希望エリア、予算、間取り、入居時期
      → 顧客マスタに自動登録（重複チェック: 電話番号 or メールアドレス）
      → 初回自動返信（5分以内: 受付確認 + 類似物件3件提案）
      → 担当者への通知（Slack/LINE WORKS）

■ 物件マッチングアルゴリズム

  スコアリング:
    match_score = Σ(wi × si)

    要素と重み:
    ┌──────────┬────┬──────────────────────┐
    │ 要素      │ 重み│ スコア算出              │
    ├──────────┼────┼──────────────────────┤
    │ エリア    │ 0.30│ 希望エリア内=1.0, 隣接=0.5, 遠方=0.0 │
    │ 賃料/価格 │ 0.25│ 予算内=1.0, 10%超=0.7, 20%超=0.3    │
    │ 間取り    │ 0.15│ 一致=1.0, ±1部屋=0.7              │
    │ 面積      │ 0.10│ 希望面積±10%=1.0, ±20%=0.5        │
    │ 駅距離    │ 0.10│ 希望以内=1.0, 超過分×0.05減         │
    │ 築年数    │ 0.05│ 希望以内=1.0, 超過5年ごとに0.1減     │
    │ 設備      │ 0.05│ 必須設備一致率                      │
    └──────────┴────┴──────────────────────┘

    match_score ≥ 0.7 → 自動提案対象
    match_score ≥ 0.5 → 追加候補として提示

■ 追客フロー自動化

  顧客温度感スコアリング:
    hot（80-100点）:  内見予約済み / 申込み検討中
    warm（50-79点）:  条件変更中 / 比較検討中
    cool（20-49点）:  情報収集段階 / 反応薄い
    cold（0-19点）:   2週間以上無反応

    スコア加算ルール:
      反響送信:         +30
```

```
      返信あり:          +20
      内見予約:          +30
      内見実施:          +10
      2回目内見:         +15
      申込み意思表示:    +20
      条件交渉:          +10
      日が経過:          -2/日（最大-30）
      メール開封:       +5
      物件ページ閲覧:   +3/回

  温度感別自動アクション:
    hot:  即日フォロー（担当者通知 + 申込書送付準備）
    warm: 3日ごとに類似物件提案メール（LLM生成）
    cool: 7日ごとに新着物件通知
    cold: 30日後に掘り起こしメール → 反応なければアーカイブ
```

## 6.2 データモデル

```
-- 顧客マスタ
CREATE TABLE realestate_customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name TEXT NOT NULL,
  name_kana TEXT,
  phone TEXT,
  email TEXT,
  line_id TEXT,
  source TEXT,                            -- suumo / homes / athome / hp / line / phone /
walk_in
  transaction_type TEXT,                  -- buy / rent
  preferences JSONB DEFAULT '{}',         -- {area: [], budget_min, budget_max, floor_plan:
[], min_area, max_station_distance, required_facilities: []}
  temperature_score INTEGER DEFAULT 30,   -- 顧客温度感スコア 0-100
  temperature_label TEXT DEFAULT 'cool',  -- hot / warm / cool / cold
  assigned_to TEXT,                       -- 担当者
  status TEXT DEFAULT 'active',           -- active / contracted / lost / archived
  last_contact_date DATE,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 内見記録
CREATE TABLE viewing_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  customer_id UUID NOT NULL REFERENCES realestate_customers(id),
  property_address TEXT NOT NULL,
  viewing_date DATE NOT NULL,
  viewing_time TEXT,                      -- HH:MM
  feedback TEXT,                          -- 内見後の感想
  interest_level INTEGER,                 -- 1-5
  next_action TEXT,                       -- follow_up / second_viewing / application /
pass
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 追客アクション履歴
CREATE TABLE follow_up_actions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  customer_id UUID NOT NULL REFERENCES realestate_customers(id),
  action_type TEXT NOT NULL,              -- auto_email / manual_call / viewing_booking /
proposal / follow_up_email
  action_date TIMESTAMPTZ NOT NULL,
  content TEXT,                           -- 送信内容 or メモ
  result TEXT,                            -- sent / opened / replied / no_response
  properties_proposed JSONB DEFAULT '[]',  -- 提案した物件リスト
  created_at TIMESTAMPTZ DEFAULT now()
```

```
);

-- RLS
ALTER TABLE realestate_customers ENABLE ROW LEVEL SECURITY;
ALTER TABLE viewing_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE follow_up_actions ENABLE ROW LEVEL SECURITY;
```

## 6.3 パイプラインステップ

| Step | マイクロエージェント | 入力 | 処理 | 出力 |
|---|---|---|---|---|
| 1 | run_structured_extractor | inquiry_email/form | 反響メール/フォームの構造化（顧客名、連絡先、希望条件抽出） | structured_inquiry |
| 2 | run_rule_matcher | structured_inquiry | 既存顧客との重複チェック（電話番号/メール）・新規登録 | customer_record |
| 3 | run_rule_matcher | customer_record + property_db | 希望条件×物件DBでマッチング（加重スコアリング） | matched_properties |
| 4 | run_message_drafter | matched_properties | 提案メール/LINE文面のLLM生成（類似物件3件提案） | proposal_message |
| 5 | run_cost_calculator | customer_record + actions | 顧客温度感スコア更新（加算/減算ルール適用） | updated_temperature |
| 6 | run_saas_writer | updated_temperature + proposal_message | 次回アクション（追客）のスケジューリング+DB保存 | scheduled_action |

## 6.4 パイプライン（実装）

```
# workers/bpo/realestate/pipelines/crm_pipeline.py

class CrmPipeline:
    """
    内見・顧客管理（CRM）パイプライン

    Step 1: inquiry_parser       — 反響メール/フォームの構造化
    Step 2: customer_matcher     — 既存顧客との重複チェック・登録
    Step 3: property_matcher     — 希望条件×物件DBでマッチング
    Step 4: proposal_generator   — 提案メール/LINE文面のLLM生成
    Step 5: temperature_updater  — 顧客温度感スコア更新
    Step 6: action_scheduler     — 次回アクション（追客）のスケジューリング
    """
```

---

# 7. 修繕・設備管理

## 7.1 緊急度判定ルール

```
■ 緊急度4段階判定

  Level 1: 緊急（2時間以内に対応開始）
    トリガーキーワード: 漏水, 水漏れ, ガス漏れ, ガス臭い, 停電, 断水,
                        トイレ詰まり（唯一のトイレ）, 鍵紛失（帰宅不能）,
                        火災報知器鳴動, 窓ガラス割れ（防犯）
    アクション: 緊急業者手配 + オーナー即時通知 + 入居者へ応急対応指示

  Level 2: 至急（24時間以内に対応開始）
    トリガーキーワード: エアコン故障（真夏/真冬）, 給湯器故障, 排水不良,
                        インターホン故障, オートロック故障, エレベータ停止
    アクション: 翌営業日までに業者手配 + オーナー通知

  Level 3: 通常（1週間以内に対応）
    トリガーキーワード: 壁紙剥がれ, 床のきしみ, 蛇口のパッキン,
                        網戸の破れ, 換気扇の異音, 照明器具の交換
    アクション: 業者見積取得 → オーナー承認 → 施工手配

  Level 4: 定期/美観（次回点検時 or 退去時）
    トリガーキーワード: 外壁の汚れ, 共用部の美観, 植栽の手入れ,
                        駐輪場の整理, 掲示板の更新
    アクション: 定期点検タスクに追加

■ AI判定ロジック
  Step 1: テキスト分類（キーワードマッチング）
    キーワード辞書で初期分類 → confidence = 0.8〜1.0

  Step 2: 写真分析（Vision AI、Phase 2+）
    修繕箇所の写真 → 損傷程度を判定 → 緊急度の補正

  Step 3: コンテキスト補正
    季節: エアコン故障は真夏(7-9月)・真冬(12-2月) → Level 2に昇格
    時間帯: 夜間(22:00-6:00)の漏水 → Level 1（応急対応必須）
    入居者属性: 高齢者・障害者 → 1段階昇格
```

## 7.2 長期修繕計画の計算式

```
■ 長期修繕計画（30年計画）

  建物部位別の修繕周期と概算費用（RC造マンション基準）：

  ┌──────────────┬──────────┬──────────────┬──────────┐
  │ 部位         │ 修繕周期│ 概算費用/㎡  │ 備考     │
  ├──────────────┼──────────┼──────────────┼──────────┤
  │ 屋上防水     │ 12-15年│ ¥4,000-8,000│ 塗膜/シート│
  │ 外壁塗装     │ 12-15年│ ¥3,000-6,000│ 足場込み   │
  │ 外壁タイル補修 │ 12-15年│ ¥2,000-5,000│ 打診検査含│
  │ 鉄部塗装     │ 5-7年  │ ¥500-1,500  │ 階段・手摺│
  │ 給水管更新   │ 25-30年│ ¥8,000-15,000│ 共用部   │
  │ 排水管更新   │ 25-30年│ ¥6,000-12,000│ 共用部   │
  │ エレベータ更新 │ 25-30年│ ¥300-500万/基│ 全面更新 │
  │ 機械式駐車場 │ 20-25年│ ¥80-150万/台 │ 全面更新 │
  │ 共用部照明LED化│ 10-15年│ ¥500-1,500  │ 一括交換 │
  │ 消防設備更新 │ 20-25年│ ¥1,000-3,000│ 法定点検必須│
  └──────────────┴──────────┴──────────────┴──────────┘

  年間修繕積立金の適正額（国交省ガイドライン）：
    15階未満・5,000㎡未満: ¥218/㎡/月（目安）
    15階未満・5,000-10,000㎡: ¥202/㎡/月
    20階以上: ¥255/㎡/月

  修繕計画シミュレーション:
    年次修繕費予測 = Σ(対象部位の面積 × ㎡単価) ※修繕周期年に発生
    累計修繕積立金 = 月額積立金 × 12 × 経過年数
    過不足判定:
      累計積立金 ≥ 累計修繕費見込み → 適正
      累計積立金 < 累計修繕費見込みの80% → 警告（増額提案）
      累計積立金 < 累計修繕費見込みの60% → 危険（緊急増額提案）
```

## 7.3 データモデル

```
-- 修繕依頼
CREATE TABLE repair_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  property_id UUID NOT NULL REFERENCES managed_properties(id),
  tenant_id UUID REFERENCES tenants(id),
  room_number TEXT,
  request_date TIMESTAMPTZ DEFAULT now(),
  description TEXT NOT NULL,                -- 依頼内容
  photo_urls JSONB DEFAULT '[]',            -- 写真URL
  urgency_level INTEGER NOT NULL,           -- 1=緊急, 2=至急, 3=通常, 4=定期
  urgency_auto_detected BOOLEAN DEFAULT TRUE, -- AI判定 or 手動
  category TEXT,                            -- plumbing / electrical / hvac / structural /
cosmetic / equipment
  assigned_vendor_id UUID,                  -- 手配業者
  estimated_cost INTEGER,                   -- 見積金額
  actual_cost INTEGER,                      -- 実際の費用
  owner_approved BOOLEAN DEFAULT FALSE,     -- オーナー承認
  status TEXT DEFAULT 'received',           -- received / assessing / vendor_assigned /
quoted / owner_pending / in_progress / completed / cancelled
  completed_date DATE,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 長期修繕計画
CREATE TABLE repair_plans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  property_id UUID NOT NULL REFERENCES managed_properties(id),
  plan_year INTEGER NOT NULL,               -- 計画年度
  component TEXT NOT NULL,                  -- 部位（屋上防水、外壁塗装等）
  planned_cost INTEGER NOT NULL,            -- 計画費用
  actual_cost INTEGER,                      -- 実績費用
  last_repair_year INTEGER,                 -- 前回実施年
  next_repair_year INTEGER,                 -- 次回予定年
  repair_cycle_years INTEGER,               -- 修繕周期（年）
  status TEXT DEFAULT 'planned',            -- planned / budgeted / in_progress / completed /
deferred
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE repair_requests ENABLE ROW LEVEL SECURITY;
ALTER TABLE repair_plans ENABLE ROW LEVEL SECURITY;
```

## 7.4 パイプラインステップ

| Step | マイクロエージェント | 入力 | 処理 | 出力 |
|---|---|---|---|---|
| 1 | run_structured_extractor | repair_request (text + photos) | 修繕依頼の構造化（テキスト+写真からカテゴリ・箇所・状況を抽出） | structured_request |
| 2 | run_rule_matcher | structured_request | 緊急度AI判定（キーワードマッチ+季節/時間帯/入居者属性コンテキスト補正） | urgency_classified |
| 3 | run_rule_matcher | urgency_classified + vendor_db | 協力業者マッチング（地域×工種×空き×過去評価） | matched_vendor |
| 4 | run_message_drafter | matched_vendor | 見積依頼の自動送信（業者向けメール/FAX文面生成） | estimate_request |
| 5 | run_message_drafter | estimate_request | オーナーへの承認依頼（緊急時Level1-2は事後報告） | owner_notification |
| 6 | run_saas_writer | owner_notification (approved) | 施工日程調整+DB保存（入居者・業者・管理会社の三者調整） | scheduled_repair |
| 7 | run_output_validator | scheduled_repair (completed) | 完了記録・修繕履歴更新+長期修繕計画への反映 | repair_record |

## 7.5 パイプライン（実装）

```
# workers/bpo/realestate/pipelines/repair_pipeline.py

class RepairPipeline:
    """
    修繕・設備管理パイプライン

    Step 1: request_parser       — 修繕依頼の構造化（テキスト+写真）
    Step 2: urgency_classifier   — 緊急度AI判定（キーワード+コンテキスト）
    Step 3: vendor_matcher       — 協力業者マッチング（地域×工種×空き）
    Step 4: estimate_request     — 見積依頼の自動送信
    Step 5: owner_notification   — オーナーへの承認依頼（緊急時は事後報告）
    Step 6: schedule_coordinator — 施工日程調整
    Step 7: completion_recorder  — 完了記録・修繕履歴更新
    """
```

# 8. 免許・届出管理

## 8.1 宅建業免許更新チェックリスト

```
■ 宅建業免許の基本情報
  – 有効期間: 5年（宅建業法3条2項）
  – 更新申請期間: 有効期間満了の90日前〜30日前（宅建業法3条3項）
  – 申請先:
    1つの都道府県のみに事務所 → 都道府県知事免許
    2つ以上の都道府県に事務所 → 国土交通大臣免許

■ 更新申請必要書類チェックリスト
  □ 免許申請書（様式第一号）
  □ 添付書類
    □ 宅地建物取引業経歴書
    □ 誓約書
    □ 専任の宅地建物取引士設置証明書
    □ 相談役・顧問・株主一覧
    □ 事務所の写真（外観、内部、表示板）
    □ 略歴書（役員全員分）
    □ 身分証明書（役員全員分 – 本籍地の市区町村発行）
    □ 登記されていないことの証明書（役員全員分 – 法務局発行）
    □ 納税証明書（法人税 or 所得税）
    □ 貸借対照表・損益計算書
    □ 宅地建物取引士証の写し（専任の宅建士全員分）
  □ 手数料
    知事免許: ¥33,000
    大臣免許: ¥33,000 + 登録免許税¥90,000

■ アラートスケジュール
  満了6ヶ月前:  「免許更新の準備を開始してください」+ チェックリスト送付
  満了90日前:   「更新申請受付開始です。書類を揃えてください」
  満了60日前:   「申請期限まで30日です。未提出書類: [リスト]」
  満了30日前:   「本日が申請期限です！」（最終警告）
  ※ 期限を過ぎると免許失効 → 無免許営業（宅建業法12条違反: 3年以下の懲役 or 300万円以下の罰金）
```

## 8.2 業務状況報告書の自動生成

```
■ 業務状況報告書（宅建業法施行規則17条の2の2）
  提出期限: 毎年度終了後50日以内（5月20日前後）
  提出先: 免許権者（都道府県知事 or 国土交通大臣）

■ 記載事項（自動集計ロジック）
  1. 事務所に関する事項
     → 事務所マスタから自動取得

  2. 宅地建物取引士に関する事項
     → 従業者マスタから宅建士を抽出
     → 専任/一般の区別
     → 登録番号・登録年月日

  3. 取引の実績
     売買:
       自己物件の売却件数・金額
       媒介・代理の件数・金額
     賃貸:
       自己物件の賃貸件数
       媒介・代理の件数
     → contract_projects + transaction_records から自動集計

  4. 兼業の状況（不動産業以外）

  5. 資産の状況（BS要約）

■ 従業者名簿（宅建業法48条3項）
  法定保存期間: 退職後10年
  記載事項:
    - 氏名
    - 生年月日
    - 従業者証明書番号
    - 主たる職務内容
    - 宅建士であるか否か
    - 就職・退職年月日
  → 従業者マスタから自動生成
  → 退職者も10年間保持（自動アーカイブ、削除防止）
```

## 8.3 データモデル

```
-- 免許・届出管理
CREATE TABLE license_management (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  license_type TEXT NOT NULL,               -- takken（宅建業免許）/ kanri（管理業登録）/ other
  license_number TEXT NOT NULL,             -- 免許番号（例: 東京都知事(5)第12345号）
  issuer TEXT NOT NULL,                     -- 免許権者
  issue_date DATE NOT NULL,                 -- 免許日
  expiry_date DATE NOT NULL,               -- 有効期限
  renewal_count INTEGER DEFAULT 1,          -- 更新回数
  renewal_status TEXT DEFAULT 'active',     -- active / renewal_pending / expired
  last_renewal_date DATE,
  checklist JSONB DEFAULT '{}',             -- 更新書類チェックリスト
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 従業者マスタ
CREATE TABLE realestate_employees (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name TEXT NOT NULL,
  birth_date DATE,
  employee_cert_number TEXT,                -- 従業者証明書番号
  is_takkenshi BOOLEAN DEFAULT FALSE,       -- 宅建士か
  takkenshi_number TEXT,                    -- 宅建士登録番号
  takkenshi_expiry DATE,                    -- 宅建士証有効期限
  is_senin BOOLEAN DEFAULT FALSE,           -- 専任の宅建士か
  office_name TEXT,                         -- 所属事務所
  primary_duty TEXT,                        -- 主たる職務内容
  hire_date DATE NOT NULL,
  termination_date DATE,                    -- 退職日
  status TEXT DEFAULT 'active',             -- active / terminated
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 届出・報告の提出履歴
CREATE TABLE regulatory_filings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  filing_type TEXT NOT NULL,                -- business_report（業務状況報告書）/
employee_roster（従業者名簿）/ change_notification（変更届）
  fiscal_year INTEGER,
  due_date DATE NOT NULL,                   -- 提出期限
  submitted_date DATE,                      -- 提出日
  filing_data JSONB DEFAULT '{}',           -- 報告書データ
  file_url TEXT,                            -- 生成PDF URL
  status TEXT DEFAULT 'pending',            -- pending / generated / reviewed / submitted
  created_at TIMESTAMPTZ DEFAULT now()
);
```

```
-- RLS
ALTER TABLE license_management ENABLE ROW LEVEL SECURITY;
ALTER TABLE realestate_employees ENABLE ROW LEVEL SECURITY;
ALTER TABLE regulatory_filings ENABLE ROW LEVEL SECURITY;
```

## 8.4 パイプラインステップ

| Step | マイクロエージェント | 入力 | 処理 | 出力 |
|---|---|---|---|---|
| 1 | run_saas_reader | company_id | 免許情報の取得・残存期間計算 | license_status |
| 2 | run_rule_matcher | license_status | 更新アラート生成（6ヶ月/90日/60日/30日前） | deadline_alerts |
| 3 | run_diff_detector | deadline_alerts + checklist_db | 必要書類チェックリスト管理（未取得書類の取得方法ガイド付与） | checklist_result |
| 4 | run_document_generator | checklist_result + transaction_data | 業務状況報告書の自動生成（取引実績・宅建士情報・資産状況） | business_report |
| 5 | run_document_generator | business_report + employee_data | 法定従業者名簿の自動生成・更新（退職者10年保持） | employee_roster |
| 6 | run_output_validator | business_report + employee_roster | バリデーション（期限超過・未提出書類の警告） | validated_output |

## 8.5 パイプライン（実装）

```
# workers/bpo/realestate/pipelines/license_pipeline.py

class LicensePipeline:
    """
    免許・届出管理パイプライン

    Step 1: license_reader       — 免許情報の取得・期限チェック
    Step 2: alert_generator      — 更新アラート生成（6ヶ月/90日/60日/30日前）
    Step 3: checklist_manager    — 必要書類チェックリスト管理
    Step 4: report_generator     — 業務状況報告書の自動生成
    Step 5: employee_roster      — 従業者名簿の自動生成・更新
    Step 6: output_validator     — バリデーション
    """

    async def license_reader(self, input_data: dict) -> dict:
        """免許情報を取得し、残存期間を計算する。"""

    async def alert_generator(self, step1: dict) -> dict:
        """
        期限に基づくアラート生成。
        - 6ヶ月前: 準備開始通知
        - 90日前: 申請受付開始通知
        - 60日前: 締切30日前警告
        - 30日前: 最終警告
        宅建士証の有効期限（5年）も同時にチェック。
        """

    async def checklist_manager(self, step2: dict) -> dict:
        """
        更新に必要な書類のチェックリストを管理。
        未取得書類の取得方法ガイドを付与。
        """

    async def report_generator(self, step3: dict) -> dict:
        """
        業務状況報告書を自動生成。
        - 取引実績: contract_projects / transaction_records から集計
        - 宅建士情報: realestate_employees から抽出
        - 資産状況: freee連携（Phase 2+）or 手動入力
        """

    async def employee_roster(self, step4: dict) -> dict:
        """
        法定従業者名簿を自動生成・更新。
        退職者も10年間保持（宅建業法48条3項）。
        """

    async def output_validator(self, step5: dict) -> LicenseResult:
        """バリデーション。期限超過や未提出書類の警告。"""
```

## 8.5 法令参照

```
■ 宅地建物取引業法
  - 3条: 免許（有効期間5年、更新規定）
  - 12条: 無免許事業等の禁止
  - 15条: 宅地建物取引士の登録
  - 22条の2: 宅地建物取引士証の交付（有効期間5年）
  - 48条: 証明書の携帯等（従業者名簿の備付け義務）
  - 49条: 帳簿の備付け（取引台帳、法定保存5年 / 新築は10年）
  - 50条: 標識の掲示

■ 宅建業法施行規則
  - 17条の2の2: 業務状況報告書（毎年度提出）
  - 15条の5の3: 従業者名簿の記載事項

■ 賃貸住宅の管理業務等の適正化に関する法律（管理業法）
  - 3条: 賃貸住宅管理業の登録（200戸以上の管理は登録義務）
  - 有効期間: 5年
  - 業務管理者の配置義務
```

---

# 横断: 外部連携

```
■ Phase 1 連携
  - 国交省 不動産取引価格情報API（物件査定）
  - freee API（経理連携 — Phase 2+で本格化）
  - クラウドサイン API（電子署名 — Phase 2+）

■ Phase 2+ 連携
  - 銀行API（口座入金データ自動取込 — 入金消込自動化）
  - SUUMO/HOME'S 物件入稿API or CSV一括入稿
  - LINE Messaging API（督促・追客のLINE連携）
  - Google Calendar API（内見スケジュール同期）
  - e内容証明サービス（電子内容証明郵便）
```

---

## 制約・前提条件

```
1. REINS（レインズ）は閉鎖ネットワーク
   - 宅建業者専用、API非公開、スクレイピング禁止
   - 対応: 公開データ（国交省API）+ 自社成約データの蓄積
   - ユーザーがREINS事例を手動入力する機能を提供

2. 不動産IDの普及状況
   - 国交省が推進中（2025年〜段階的運用開始）
   - 普及後はAPI連携で物件データの名寄せが容易に
   - Phase 2+で対応予定

3. 電子契約の普及
   - 2022年5月の宅建業法改正で全面解禁
   - ただし中小不動産会社の導入率はまだ低い
   - Phase 1は紙ベース（Word/PDF出力）を前提、電子署名はオプション

4. 個人情報保護
   - 入居者情報、顧客情報は全て company_id ベースのRLS
   - PII（氏名、住所、電話番号、口座情報）はログに出力しない
   - 敷金台帳・督促履歴は法定保存期間後に自動アーカイブ

5. 仲介手数料の計算
   - 宅建業法46条の報酬上限を厳守
   - 上限超過の請求書は生成を阻止（error レベル）
```

---

## データベース設計（RLSポリシー統一定義）

各モジュールのテーブルは上記各セクションで定義済み。以下はテナント分離のためのRLSポリシーを統一定義する。

```
-- ==========================================
-- 不動産BPO テナント分離RLSポリシー
-- 全テーブル共通: company_id ベースのRLS
-- ==========================================

-- 査定系
CREATE POLICY appraisal_projects_tenant ON appraisal_projects
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY comparable_transactions_tenant ON comparable_transactions
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY transaction_records_tenant ON transaction_records
    USING (company_id = current_setting('app.company_id', true)::UUID);

-- 契約書系
CREATE POLICY contract_projects_tenant ON contract_projects
    USING (company_id = current_setting('app.company_id', true)::UUID);

-- 賃貸管理系
CREATE POLICY managed_properties_tenant ON managed_properties
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY tenants_tenant ON tenants
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY payment_records_tenant ON payment_records
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY collection_actions_tenant ON collection_actions
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY owner_remittances_tenant ON owner_remittances
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY deposit_ledger_tenant ON deposit_ledger
    USING (company_id = current_setting('app.company_id', true)::UUID);

-- CRM系
CREATE POLICY realestate_customers_tenant ON realestate_customers
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY viewing_records_tenant ON viewing_records
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY follow_up_actions_tenant ON follow_up_actions
    USING (company_id = current_setting('app.company_id', true)::UUID);

-- 修繕系
CREATE POLICY repair_requests_tenant ON repair_requests
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY repair_plans_tenant ON repair_plans
    USING (company_id = current_setting('app.company_id', true)::UUID);

-- 免許・届出系
CREATE POLICY license_management_tenant ON license_management
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY realestate_employees_tenant ON realestate_employees
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY regulatory_filings_tenant ON regulatory_filings
    USING (company_id = current_setting('app.company_id', true)::UUID);
```